실습 클래스 정답 코드 모음집

코드 전용판

주차별로 독립된 코드이며, 각 public 클래스는 같은 이름의 .java 파일에 둡니다.

문제 PDF · 주차별 소스 · 자료.zip의 ch07/08을 대조하여 정리했습니다.

03주차 이진 트리 기본 연산

높이: 빈 트리 -1, 리프 0. 수식 계산은 정수 연산.

BTNode.java

```
public class BTNode {
    String data;
    BTNode Lchild;
    BTNode Rchild;
    public BTNode() {
    }
    public BTNode(String dt) {
        data = dt;
        Lchild = null;
        Rchild = null;
    }
    public BTNode(BTNode lc, String dt, BTNode rc) {
        data = dt;
        Lchild = lc;
        Rchild = rc;
    }
}
```

BinaryTree.java

```
public class BinaryTree {
    BTNode root;
    public BinaryTree() {
        root = null;
    }
    public BinaryTree(BinaryTree ltree, String data, BinaryTree rtree) {
        root = new BTNode(ltree.root, data, rtree.root);
    }
    public boolean isEmpty() {
        return root == null;
    }
    public BinaryTree leftSubTree() {
        BinaryTree tmp = new BinaryTree();
        if (root != null) {
            tmp.root = root.Lchild;
        }
        return tmp;
    }
    public BinaryTree rightSubTree() {
        BinaryTree tmp = new BinaryTree();
        if (root != null) {
            tmp.root = root.Rchild;
        }
        return tmp;
    }
```

3주차 | BinaryTree.java · 계속

```
    public String rootData() {
        if (root == null) {
            System.out.println("빈루트");
            return null;
        }
        return root.data;
    }
    public int calculate() {
        if (root == null) {
            throw new IllegalStateException("Empty expression tree");
        }
        return theCalculate(root);
    }
    private int theCalculate(BTNode t) {
        if (t.Lchild == null && t.Rchild == null) {
            return Integer.parseInt(t.data);
        }
        int left = theCalculate(t.Lchild);
        int right = theCalculate(t.Rchild);
        switch (t.data) {
            case "+": return left + right;
            case "-": return left - right;
            case "*": return left * right;
            case "/": return left / right;
            default:
                throw new IllegalArgumentException("Unknown operator");
        }
    }
    public int height() {
        return theHeight(root);
    }
    private int theHeight(BTNode t) {
        if (t == null) {
            return -1;
        }
        int leftHeight = theHeight(t.Lchild);
        int rightHeight = theHeight(t.Rchild);
        return 1 + Math.max(leftHeight, rightHeight);
    }
    public void print0() {
        print0(root);
        System.out.println();
    }
    private void print0(BTNode t) {
        if (t == null) return;
        if (t.Lchild == null && t.Rchild == null) {
            System.out.print(t.data);
            return;
        }
        System.out.print("(");
        print0(t.Lchild);
        System.out.print(t.data);
        print0(t.Rchild);
        System.out.print(")");
    }
}
```

05주차 순회 · 복사 · 동등 비교

Stack·Queue는 제공된 보조 클래스이며 뒤에 함께 수록.

BTNode.java

```
public class BTNode {
    String data;
    BTNode Lchild;
    BTNode Rchild;
    public BTNode() {
    }
    public BTNode(String dt) {
        data = dt;
        Lchild = null;
        Rchild = null;
    }
    public BTNode(BTNode lc, String dt, BTNode rc) {
        data = dt;
        Lchild = lc;
        Rchild = rc;
    }
}
```

BinaryTree.java

```
public class BinaryTree {
    BTNode root;
    public BinaryTree() {
        root = null;
    }
    public BinaryTree(BinaryTree ltree, String data, BinaryTree rtree) {
        root = new BTNode(ltree.root, data, rtree.root);
    }
    public boolean isEmpty() {
        return root == null;
    }
    public BinaryTree leftSubTree() {
        BinaryTree ltree = new BinaryTree();
        ltree.root = root.Lchild;
        return ltree;
    }
    public BinaryTree rightSubTree() {
        BinaryTree rtree = new BinaryTree();
        rtree.root = root.Rchild;
        return rtree;
    }
    public String rootData() {
        return root.data;
    }
    public void inorder() {
        System.out.println();
        System.out.println("Inorder");
        theInorder(root);
        System.out.println();
        System.out.println("InorderIter");
        inorderIter1();
    }
```

5주차 | BinaryTree.java · 계속

```
    public void preorder() {
        System.out.println("preorder");
        System.out.println();
        thePreorder(root);
        System.out.println();
        System.out.println("preorderIter");
        preorderIter();
    }
    public void postorder() {
        thePostorder(root);
    }
    private void theInorder(BTNode t) {
        if (t != null) {
            theInorder(t.Lchild);
            System.out.print(t.data);
            theInorder(t.Rchild);
        }
    }
    private void thePreorder(BTNode t) {
        if (t != null) {
            System.out.print(t.data);
            thePreorder(t.Lchild);
            thePreorder(t.Rchild);
        }
    }
    private void thePostorder(BTNode t) {
        if (t != null) {
            thePostorder(t.Lchild);
            thePostorder(t.Rchild);
            System.out.print(t.data);
        }
    }
    private void inorderIter1() {
        Stack stack = new Stack();
        BTNode current = root;
        while (current != null || !stack.empty()) {
            while (current != null) {
                stack.push(current);
                current = current.Lchild;
            }
            current = (BTNode) stack.pop();
            System.out.print(current.data);
            current = current.Rchild;
        }
    }
```

5주차 | BinaryTree.java · 계속

```
    private void preorderIter() {
        if (root == null)
            return;
        Stack stack = new Stack();
        stack.push(root);
        while (!stack.empty()) {
            BTNode current = (BTNode) stack.pop();
            System.out.print(current.data);
            if (current.Rchild != null)
                stack.push(current.Rchild);
            if (current.Lchild != null)
                stack.push(current.Lchild);
        }
    }
    public void levelorder() {
        if (root == null)
            return;
        Queue queue = new Queue();
        queue.enqueue(root);
        while (!queue.isEmpty()) {
            BTNode current = (BTNode) queue.dequeue();
            System.out.print(current.data);
            if (current.Lchild != null)
                queue.enqueue(current.Lchild);
            if (current.Rchild != null)
                queue.enqueue(current.Rchild);
        }
    }
    public BinaryTree copy() {
        BinaryTree newTree = new BinaryTree();
        newTree.root = theCopy(root);
        return newTree;
    }
    private BTNode theCopy(BTNode t) {
        if (t == null)
            return null;
        BTNode newNode = new BTNode(
            theCopy(t.Lchild), t.data, theCopy(t.Rchild));
        return newNode;
    }
    public boolean equals(BinaryTree tr) {
        return theEqual(this.root, tr.root);
    }
    private boolean theEqual(BTNode s, BTNode t) {
        if (s == null && t == null)
            return true;
        if (s == null || t == null)
            return false;
        if (!s.data.equals(t.data))
            return false;
        return theEqual(s.Lchild, t.Lchild)
            && theEqual(s.Rchild, t.Rchild);
    }
}
```

Stack.java

```java
public class Stack {
    private int top;
    private Object[] data;
    private final static int MAX = 100;
    public Stack() {
        data = new Object[MAX];
        top = -1;
    }
    public int size() {
        return top + 1;
    }
    public boolean empty() {
        return top == -1;
    }
    public void push(Object x) {
        top = top + 1;
        data[top] = x;
    }
    public Object pop() {
        top = top - 1;
        return data[top + 1];
    }
    public Object peek() {
        return data[top];
    }
}
```

Queue.java

```java
public class Queue {
    private int front;
    private int back;
    private Object[] data;
    private final static int MAX = 100;
    public Queue() {
        data = new Object[MAX];
        front = back = 0;
    }
    public int size() {
        if (back >= front)
            return back - front;
        else
            return MAX + back - front;
    }
    public boolean isEmpty() {
        return front == back;
    }
    public void enqueue(Object x) {
        back = (back + 1) % MAX;
        if (back == front) {
        }
        data[back] = x;
    }
    public Object dequeue() {
        front = (front + 1) % MAX;
        return data[front];
    }
```

5주차 | Queue.java · 계속

```
    public Object front() {
        return data[(front + 1) % MAX];
    }
}
```

07주차 이진 탐색 트리

BSTsplit은 강의의 연결 변경 방식. 분할 후 원본은 빈 트리.

TreeNode.java

```
public class TreeNode {
    String key;
    TreeNode Lchild;
    TreeNode Rchild;
}
```

BinarySearchTree.java

```
public class BinarySearchTree {
    private TreeNode rootNode;
    public TreeNode BSTsearch(String K) {
        TreeNode p = rootNode;
        while (p != null) {
            int result = K.compareTo(p.key);
            if (result == 0)
                return p;
            else if (result < 0)
                p = p.Lchild;
            else
                p = p.Rchild;
        }
        return null;
    }
```

7주차 | BinarySearchTree.java · 계속

```
    public void BSTinsert(String K) {
        TreeNode newNode = new TreeNode();
        newNode.key = K;
        newNode.Lchild = null;
        newNode.Rchild = null;
        if (rootNode == null) {
            rootNode = newNode;
            return;
        }
        TreeNode p = rootNode;
        TreeNode parent = null;
        while (p != null) {
            parent = p;
            int result = K.compareTo(p.key);
            if (result == 0)
                return;
            else if (result < 0)
                p = p.Lchild;
            else
                p = p.Rchild;
        }
        if (K.compareTo(parent.key) < 0)
            parent.Lchild = newNode;
        else
            parent.Rchild = newNode;
    }
    private TreeNode maxNode(TreeNode root) {
        if (root == null)
            return null;
        while (root.Rchild != null) {
            root = root.Rchild;
        }
        return root;
    }
    private TreeNode delete(TreeNode root, String K) {
        if (root == null)
            return null;
        int result = K.compareTo(root.key);
        if (result < 0) {
            root.Lchild = delete(root.Lchild, K);
        } else if (result > 0) {
            root.Rchild = delete(root.Rchild, K);
        } else {
            if (root.Lchild == null)
                return root.Rchild;
            if (root.Rchild == null)
                return root.Lchild;
            TreeNode temp = maxNode(root.Lchild);
            root.key = temp.key;
            root.Lchild = delete(root.Lchild, temp.key);
        }
        return root;
    }
    public void BSTdelete(String K) {
        rootNode = delete(rootNode, K);
    }
```

7주차 | BinarySearchTree.java · 계속

```
    public boolean BSTsplit(String x,
                            BinarySearchTree SmallTree,
                            BinarySearchTree LargeTree) {
        TreeNode Small = new TreeNode();
        TreeNode Large = new TreeNode();
        TreeNode S = Small;
        TreeNode L = Large;
        TreeNode P = rootNode;
        boolean found = false;
        while (P != null) {
            int result = x.compareTo(P.key);
            if (result == 0) {
                S.Rchild = P.Lchild;
                L.Lchild = P.Rchild;
                found = true;
                break;
            } else if (result < 0) {
                L.Lchild = P;
                L = P;
                P = P.Lchild;
            } else {
                S.Rchild = P;
                S = P;
                P = P.Rchild;
            }
        }
        if (!found) {
            S.Rchild = null;
            L.Lchild = null;
        }
        SmallTree.rootNode = Small.Rchild;
        LargeTree.rootNode = Large.Lchild;
        rootNode = null;
        return found;
    }
    private void printNode(TreeNode n) {
        if (n != null) {
            System.out.print("(");
            printNode(n.Lchild);
            System.out.print(" " + n.key + " ");
            printNode(n.Rchild);
            System.out.print(")");
        }
    }
    public void print() {
        printNode(rootNode);
        System.out.println();
    }
}
```

08주차 최대 힙

실습 출력 기준으로 입력 배열 origArray도 1번부터 사용(0번 제외).

문자열 compareTo()로 비교. 배열 생성자는 아래에서 위로 힙화.

Heap.java

```
public class Heap {
    private int count;
    private int size;
    private String[] itemArray;
    public Heap() {
        count = 0;
        size = 64;
        itemArray = new String[size];
    }
    public Heap(String[] origArray) {
        count = Math.max(0, origArray.length - 1);
        size = Math.max(64, count + 1);
        itemArray = new String[size];
        for (int i = 1; i <= count; i++) {
            itemArray[i] = origArray[i];
        }
        for (int i = count / 2; i >= 1; i--) {
            int parent = i;
            int child = parent * 2;
            String temp = itemArray[parent];
            while (child <= count) {
                if (child < count &&
                    itemArray[child].compareTo(itemArray[child + 1]) < 0) {
                    child++;
                }
                if (temp.compareTo(itemArray[child]) >= 0) {
                    break;
                }
                itemArray[parent] = itemArray[child];
                parent = child;
                child = parent * 2;
            }
            itemArray[parent] = temp;
        }
    }
    public void insert(String newItem) {
        if (count + 1 >= size) {
            size *= 2;
            String[] temp = new String[size];
            System.arraycopy(itemArray, 0, temp, 0, count + 1);
            itemArray = temp;
        }
        int i = ++count;
        while (i > 1 && itemArray[i / 2].compareTo(newItem) < 0) {
            itemArray[i] = itemArray[i / 2];
            i /= 2;
        }
        itemArray[i] = newItem;
    }
```

8주차 | Heap.java · 계속

```
    public String delete() {
        if (count == 0) {
            return null;
        }
        String max = itemArray[1];
        String last = itemArray[count];
        itemArray[count] = null;
        count--;
        int parent = 1;
        int child = 2;
        while (child <= count) {
            if (child < count &&
                itemArray[child].compareTo(itemArray[child + 1]) < 0) {
                child++;
            }
            if (last.compareTo(itemArray[child]) >= 0) {
                break;
            }
            itemArray[parent] = itemArray[child];
            parent = child;
            child = parent * 2;
        }
        if (count > 0) {
            itemArray[parent] = last;
        }
        return max;
    }
}
```